

스스로 판단하고 작업하는 멀티에이전트 만들기

지금까지 만든 멀티에이전트는 작업 하나가 끝날 때마다 이어서 무슨 작업을 해야 할지 사용자에게 물어봐야 했습니다. 이 장에서는 에이전트들이 공유할 수 있는 공동 목표를 설정하고 에이전트마다 그 목표를 달성하고 있는지 스스로 평가해서 다음 작업을 진행하도록 개선하겠습니다. 목표를 달성했거나 에이전트가 스스로 판단하기 어려운 상황일 때는 사용자에게 질문하는 방식으로 프로그램을 발전시키겠습니다.



15-1 에이전트의 공동 목표 만들기

15-2 템플릿으로 더 명확한 가이드 세우기

15-3 스스로 리뷰하고 수정하는 에이전트로 발전시키기

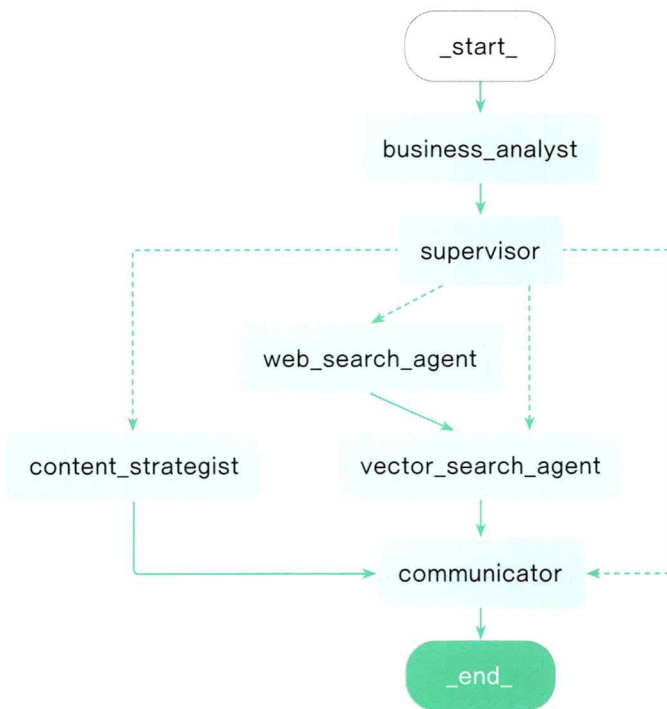


15-1 에이전트의 공동 목표 만들기

목표를 점검하는 비즈니스 분석가 에이전트 `business_analyst`를 만들고, 사용자의 의도를 파악해 에이전트의 공동 목표를 설정해 보겠습니다.

목표를 점검하는 비즈니스 분석가 에이전트

지금까지 각 AI 에이전트는 작업을 완료한 후 자신의 작업 내용을 다른 AI 에이전트들에게 공유했지만 현재 멀티에이전트 시스템은 공동 목표가 모호하고 다음 작업을 위한 판단 기준도 애매합니다. 이런 상황은 여러 사람이 함께 일을 할 때 종종 일어납니다. 각자 맡은 일이 왜 중요한지, 잘 진행되고 있는지, 수정이 필요한지 등의 목표를 점검하는 기준이 없다면 조직의 힘은 분산되고 시너지가 나지 않습니다. 책의 목차를 만드는 프로그램에서도 이와 같은 문제가 있습니다. 이번 실습에서는 이 문제를 해결하기 위해 `supervisor`가 일을 분배하기 전에 목표를 세우는 비즈니스 분석가 에이전트 `business_analyst`를 만들겠습니다. `business_analyst`는 사용자의 요구 사항과 작업의 진행 상황을 분석하여 현재 작업의 목표와 방법을 제시하는 역할을 합니다.



비즈니스 분석가 에이전트인 `business_analyst`의 그래프 구조

Do it! 실습 사용자의 의도를 파악하는 에이전트 business_analysist 만들기

■ 결과 파일: sec01/book_writer.py, models.py

비즈니스 분석가 에이전트 business_analyst에게 사용자가 입력한 내용을 바탕으로 사용자의 의도를 1차로 분석하는 역할을 맡기겠습니다. 이전까지는 슈퍼바이저 에이전트 supervisor가 이 역할까지 수행했지만 이제 supervisor는 일 분배에 집중하고 business_analysist가 현재 사용자의 요구 사항을 파악하는 데 집중하도록 만들겠습니다.

1. State에 사용자의 요구 사항이 무엇인지 담아 둘 user_request를 마련합니다. 이 값은 business_analysist가 사용자와 대화한 내용과 현재의 진행 상황(목차, 참고 자료)을 바탕으로 분석하여 채울 것입니다.

사용자의 요구 사항을 분석하는 business_analysist 추가하기

■ book_writer.py

```
(... 생략 ...)  
class State(TypedDict):  
    messages: List[AnyMessage | str]  
    task_history: List[Task]  
    references: dict # vector search agent에서 검색한 정보를 저장하는 변수  
    user_request: str # 사용자의 요구 사항을 저장하는 변수  
  
def business_analysist(state: State):  
    print("\n\n===== BUSINESS ANALYST =====")  
  
    business_analysist_system_prompt = PromptTemplate.from_template(  
        """  
        너는 책을 쓰는 AI 팀의 비즈니스 애널리스트로서,  
        AI 팀의 진행상황과 "사용자 요구 사항"을 토대로,  
        현 시점에서 '지난 요구 사항(previous_user_request)'과 최근 사용자의 발언을  
        바탕으로 요구사항이 무엇인지 판단한다.  
        지난 요구 사항이 달성되었는지 판단하고, 현 시점에서 어떤 작업을 해야 하는지 결정한다.  
  
        다음과 같은 템플릿 형태로 반환한다.  
        ```  
 - 목표: 0000 \n 방법: 0000
        ```  
  
        -----  
        *지난 요구 사항(previous_user_request)* : {previous_user_request}  
        -----  
        사용자 최근 발언: {user_last_comment}  
        -----  
        """  
    )
```

2

```

참고자료: {references}
-----
목차 (outline): {outline}
-----
"messages": {messages}
"""
)

```

2

```

ba_chain = business_analyst_system_prompt | llm | StrOutputParser()

```

3

```

#상태에서 메시지 가져오기
messages = state["messages"]

#사용자의 마지막 발언 가져오기
user_last_comment = None
for m in messages[::-1]:
    if isinstance(m, HumanMessage):
        user_last_comment = m.content
        break

```

4

```

#입력 자료 준비
inputs = {
    "previous_user_request": state.get("user_request", None),
    "references": state.get("references", {"queries": [], "docs": []}),
    "outline": get_outline(current_path),
    "messages": messages,
    "user_last_comment": user_last_comment
}

```

```

user_request = ba_chain.invoke(inputs)

```

5

```

business_analyst_message = f"[Business Analyst] {user_request}"
print(business_analyst_message)
messages.append(AIMessage(business_analyst_message))

```

6

```

save_state(current_path, state)

```

7

```

return {
    "messages": messages,
    "user_request": user_request
}

```

(... 생략 ...)

- 1 business_anaylist라는 노드를 추가하기 위해 함수를 만듭니다.
- 2 시스템 프롬프트를 작성합니다. business_anaylist는 목표를 위해 현 시점에서 어떤 작업을 해야 하는지 판단하는 역할입니다. 이 역할에 대해 자세히 설명하고 어떤 방식으로 답변을 생성할지 구체적으로 표현합니다. 이 판단에 사용할 자료는 지난 요구 사항, 사용자의 최근 발언, 참고 자료(벡터 검색한 결과), 현재 목차, 대화 기록입니다.
- 3 프롬프트를 언어 모델과 연결하고 문자열로 최종 출력되도록 StrOutputParser를 사용해서 ba_chain을 만듭니다.
- 4 이 체인에 입력할 인풋값을 설정합니다. 사용자의 마지막 발언을 가져오기 위해 기존 대화 내역(messages) 중 에서 가장 뒤에 있는 HumanMessage를 user_last_comment로 정합니다.
- 5 ba_chain을 이용해 목표와 방법을 프롬프트 템플릿대로 생성하는 부분입니다. 이를 통해 사용자의 요청이 무엇인지 파악해 user_request 변수에 담습니다.
- 6 business_analysis가 분석해서 도출한 user_request를 AIMessage로 만들어 기존 대화 내용을 담고 있는 messages에 추가합니다.
- 7 현재 state를 저장합니다. 원래는 사용자가 메시지를 입력한 직후에 저장했지만 앞으로는 사용자가 입력하지 않아도 알아서 루프를 여러 번 돌 수 있으므로 여기에서도 state를 저장하도록 합니다.

2. 그래프에 business_analyst 노드와 엣지를 추가합니다. 이전에는 사용자의 입력 내용이 supervisor에서 처음으로 처리되었지만, 이제는 business_analyst가 먼저 사용자의 입력을 받아서 목표가 무엇인지 판단하는 구조로 변경했습니다. 초기 state 설정도 user_request가 포함되도록 수정합니다.

그래프에 business_analyst 추가하고, State 초기값에 user_request 추가하기

book_writer.py

```
(... 생략 ...)  
# 상태 그래프 정의  
graph_builder = StateGraph(State)  
  
# 노드  
graph_builder.add_node("business_analyst", business_analyst)  
graph_builder.add_node("supervisor", supervisor)  
graph_builder.add_node("communicator", communicator)  
graph_builder.add_node("content_strategist", content_strategist)  
graph_builder.add_node("vector_search_agent", vector_search_agent)  
graph_builder.add_node("web_search_agent", web_search_agent)  
  
# 엣지  
graph_builder.add_edge(START, "business_analyst")  
graph_builder.add_edge("business_analyst", "supervisor")  
graph_builder.add_conditional_edges(  
    "supervisor",
```

```

supervisor_router,
{
    "content_strategist": "content_strategist",
    "communicator": "communicator",
    "vector_search_agent": "vector_search_agent",
    "web_search_agent": "web_search_agent"
}
)
(... 생략 ...)
# 상태 초기화
state = State(
    messages = [
        SystemMessage(
            f"""
                너희 AI들은 사용자의 요구에 맞는 책을 쓰는 작가 팀이다.
                사용자가 사용하는 언어로 대화하라.

                현재 시각은 {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}이다.

                ""
            )
    ],
    task_history=[],
    references={"queries": [], "docs": []},
    user_request=""
)

```

변경한 코드를 실행해 봅시다. 어떤 책을 쓰고 싶은지 이야기를 하자 Business_analyst가 수행 해야 할 작업의 목표와 방법을 작성해 줍니다. 그리고 그 결과를 supervisor가 받아서 목적을 작성하는 content_strategist에게 작업을 지시합니다. 상황에 따라 Business_analyst가 어떤 목표를 설정하고 supervisor가 어떤 작업을 지시할지가 달라집니다.

User: HYBE와 JYP 비교하는 책 쓰자. CEO와 경영전략에 대해 써줘.

```

===== BUSINESS ANALYST =====
[Business Analyst] ```
- 목표: HYBE와 JYP를 비교하는 책 작성
  방법: 두 회사의 CEO와 경영전략에 대한 심층 분석 및 비교
```

```

===== SUPERVISOR =====

[Supervisor] agent='content\_strategist' done=False description='HYBE와 JYP를 비교하는 책의 콘텐츠 전략을 수립하고, CEO와 경영전략에 대한 전체 책의 목차를 작성합니다.' done\_at=''

===== CONTENT STRATEGIST =====

책의 세부 목차를 구성하기 위해 사용자의 요구사항과 이전 대화 내용을 바탕으로 HYBE와 JYP를 비교하는 책의 목차를 제안하겠습니다. 이 책은 두 엔터테인먼트 대기업의 CEO와 경영전략을 중심으로 비교 분석하는 내용을 담고 있습니다.

### 목차 제안

1. \*\*서문\*\*

- 책의 목적 및 개요
- HYBE와 JYP 소개

2. \*\*HYBE의 CEO와 경영전략\*\*

- CEO 소개: 방시혁
- 경영철학과 리더십 스타일
- 주요 경영전략 및 성공 사례
- 글로벌 확장 전략

(... 생략 ...)

## 15-2 템플릿으로 더 명확한 가이드 세우기

AI 에이전트가 사용자가 원하는 대로 일을 하지 않는다면 AI 에이전트 자체의 한계일 수도 있지만 가이드가 구체적이지 않기 때문일 수 있습니다. 이번 절에서는 목차 작성을 담당하는 content\_strategist의 프롬프트를 구체화하고 그 결과를 AI 에이전트가 분석하도록 수정하겠습니다.

### 문서 양식을 정의하고 답변 형식을 유도하는 템플릿

템플릿은 언어 모델의 행동을 우리가 원하는 방식으로 유도하기 위해 구체적인 가이드를 제공하는 문서입니다. 이 템플릿을 활용해 언어 모델이 작성해야 하는 문서의 양식을 정의하고 답변 형식을 유도할 수 있습니다. 템플릿은 다음 3단계로 구성하겠습니다.

1. 목표 정의: 어떤 의도로 목차를 작성할 것인지 결정합니다.
2. 간략한 목차 작성: 간단한 목차를 작성합니다.
3. 상세한 목차 작성: 목차를 상세하게 만듭니다.

이렇게 단계를 나누는 이유는 언어 모델이 자세한 목차를 한 번에 잘 만들지 못하기 때문입니다. 언어 모델에게 처음부터 자세한 목차를 만들라고 하면 대부분 짧고 단순한 구조로 작성합니다. 반면 간단한 목차를 먼저 만들고 그걸 발전시키는 방식으로 작업하도록 유도하면 언어 모델이 제대로 작업을 완료할 가능성이 훨씬 높아집니다. 첫 번째 단계에서 어떤 의도로 목차를 작성할 것인지 먼저 결정하도록 한 것도 같은 맥락입니다. 의도가 설정되면 그에 맞춰서 뒤에 작성하는 내용도 영향을 받습니다.

물론 더 효율적인 방법이 있을 수 있습니다. 언어 모델을 비롯한 여러 생성형 AI 모델들이 급속도로도 발전하고 있으므로 점차 이런 복잡한 단계없이도 자세한 요구를 문제없이 처리할 수 있을 것입니다. 이번 실습에서 만들 템플릿은 명확한 가이드를 위한 하나의 예시로 참고하기를 바랍니다.



## Do it! 실습 목차 작성을 위한 템플릿 만들기

■ 결과 파일: sec02/templates/outline\_template.md

이제 목차 작성 템플릿을 만들어 보겠습니다. 언어 모델이 작성해야 하는 문서의 양식을 정의하고 답변 형식을 유도할 수 있도록 앞서 살펴본 3단계로 나눠 작성합니다. 템플릿의 마지막에 있는 -----: DONE :-----이라는 구분자로 목차를 작성한 후 콘텐츠 전략가 에이전트가 후기를 작성하도록 유도합니다.

■ ./templates/outline\_template.md

```
1. 목표 정의
기획 의도 및 제시하고자 하는 메시지
책의 기획 의도와 독자들에게 전달하고자 하는 메시지를 더욱 구체적으로 서술합니다.
 1. ~~~~~
 2. ~~~~~
 3.

지난 목차(previous_outline)에 비해 이번 목차 작성 시 달성하려는 목표
 1. ~~~~~
 2. ~~~~~
 3.

2. 간략한 목차 작성
보고서 제목
보고서 부제목: 독자들이 읽고 싶도록 hook의 역할을 할 수 있는 문구로 적는다.

chapter 제목 및 내용 간략 소개
Chapter 1: 제목
- 내용
Chapter 2: 제목
- 내용
..... (전체 구성을 고려하여 끝까지 작성하세요.)

3. 상세한 목차 작성
세부 목차

:---OUTLINE STARTS HERE:---:

Chapter 1: 제목
```

- **Chapter 목적:** Chapter 1의 전체 목표를 구체적으로 서술합니다.
- **Chapter 내용:** Chapter에 포함될 주제를 상세히 나열합니다.

### ### Section 1.1: 제목

- **Section 목적:** Section 1.1에서 전달할 구체적인 목표와 이유를 설명합니다.
- **Section 내용:** 다룰 구체적인 내용을 상세히 나열합니다.
- **주요 내용:**
  - 구체적인 설명과 논점 작성.
  - 필요한 세부 사례나 추가 설명 포함.
  - ...주요 내용 개수는 필요한만큼 추가할 수 있다.
- **참고 문헌:**
  1. [관련 문서 제목](https://example.com):인용하려는 내용 요약
  2. [관련 문서 제목](https://example.com):인용하려는 내용 요약
  3. [관련 문서 제목](https://example.com):인용하려는 내용 요약
  4. ...참고 문헌은 필요한 만큼 추가할 수 있으며 다다익선이다.

### ### Section 1.2: 제목

- **Section 목적:** Section 1.2에서 전달할 구체적인 목표와 이유를 설명합니다.
- **Section 내용:** 다룰 구체적인 내용을 상세히 나열합니다.
- **주요 내용:**
  - 구체적인 설명과 논점 작성.
  - 필요한 세부 사례나 추가 설명 포함.
  - ...주요 내용 개수는 필요한 만큼 추가할 수 있다.
- **참고 문헌:**
  1. [관련 문서 제목](https://example.com):인용하려는 내용 요약
  2. [관련 문서 제목](https://example.com):인용하려는 내용 요약
  3. [관련 문서 제목](https://example.com):인용하려는 내용 요약
  4. ...참고 문헌은 필요한 만큼 추가할 수 있으며 다다익선이다.

... 위와 같은 방식으로 섹션을 추가해 나가야 한다.

:---CHAPTER DIVIDER---:

## ## Chapter 2: 제목

.....

:---CHAPTER DIVIDER---:

## ## Chapter 3: 제목

.....

-----: DONE :-----

### + 목차 작성 후기

- 사용자의 요구 사항이 적절히 반영되었는지 쓴다.
- 개선할 점이 있는지 판단한다.

이 내용은 파이썬을 사용하지 않은 마크다운 문서입니다. 마치 신입 직원에게 ‘이런 식으로 작업해’라고 알려 주는 가이드 문서와 비슷한 역할입니다. 그럼 콘텐츠 전략가 에이전트가 이 문서를 활용해 작업할 수 있도록 수정해 봅시다.

## Do it! 실습 목차 작성 템플릿을 활용해 시스템 프롬프트 발전시키기

■ 결과 파일: sec02/book\_writer\_0.py

앞선 실습에서 만든 목차 작성 템플릿을 이용해 AI 에이전트에게 업무를 명확하게 지시하는 코드를 작성해 봅시다.

목차를 작성하는 데 필요한 구체적인 규칙들을 프롬프트로 추가합니다. 그리고 `business_analyst`가 설정한 `user_request`를 잘 준수하도록 프롬프트에 이를 반영합니다. `user_request`에는 `business_analyst`가 사용자의 의도를 파악하여 적어 둔 내용이 담겨 있습니다.

### 목차 작성 방식을 더 구체적으로 설명하기

■ book\_writer.py

```
(... 생략 ...)
def content_strategist(state: State):
 print("\n\n===== CONTENT STRATEGIST =====")

 task_history = state.get("task_history", [])
 task = task_history[-1]
 if task.agent != "content_strategist":
 raise ValueError(f"Content Strategist가 아닌 agent가 목차 작성을 시도하고
있습니다.\n {task}")

 content_strategist_system_prompt = PromptTemplate.from_template(
 """
 너는 책을 쓰는 AI 팀의 콘텐츠 전략가(Content Strategist)로서,
 이전 대화 내용을 바탕으로 사용자의 요구 사항을 분석하고, AI팀이 쓸 책의 세부 목차를 결정한다.

 기존 목차가 있다면 그 버전을 사용자의 요구에 맞게 수정하고, 없다면 새로운 목차를 제안한다.
 목차를 작성하는데 필요한 정보는 "참고 자료"에 있으므로 활용한다.

 다음 정보를 활용하여 목차를 작성하라.
 - 사용자 요구사항(user_request)
 - 작업(task)
 - 검색 자료(references)
```

- 기존 목차(previous\_outline)
- 이전 대화 내용(messages)

너의 작업 목표는 다음과 같다:

1. 만약 "기존 목차 구조(previous\_outline)"이 존재한다면, 사용자의 요구 사항을 토대로 "기존 목차 구조"에서 어떤 부분을 수정하거나 추가할지 결정한다.

- "이번 목차 작성의 주안점"에 사용자 요구사항(user\_request)을 충족시키는 것을 명시해야 한다.

2. 책의 전반적인 구조(chapter, section)를 설계하고, 각 chapter와 section의 제목을 정한다.

3. 책의 전반적인 세부 구조(chapter, section, sub-section)를 설계하고, sub-section 하부의 주요 내용을 리스트 형태로 정리한다.

4. 목차의 논리적인 흐름이 사용자 요구를 충족시키는지 확인한다.

5. 참고 자료(references)를 적극 활용하여 근거에 기반한 목차를 작성한다.

6. 참고 문헌은 반드시 참고 자료(references) 자료를 근거로 작성해야 하며, 최대한 풍부하게 준비한다. URL은 전체 주소를 적어야 한다.

7. 추가 자료나 리서치가 필요한 부분을 파악하여 supervisor에게 요청한다.

사용자 요구사항(user\_request)을 최우선으로 반영하는 목차로 만들어야 한다.

```

- 사용자 요구사항(user_request):
{user_request}
```

```

- 작업(task):
{task}
```

```

- 참고 자료(references)
{references}
```

```

- 기존 목차(previous_outline)
{outline}
```

```

- 이전 대화 내용(messages)
{messages}
```

작성 형식 아래 양식을 지키되 하부 항목으로 더 세분화해도 좋다. 목차(outline) 양식의 챕터, 섹션 등 항목의 개수는 필요한 만큼 추가하라.

섹션 개수는 최소 2개 이상이어야 하며, 더 많으면 좋다.

outline\_template은 예시로 앞부분만 제시한 것이다. 각 장은 ':---CHAPTER DIVIDER---:' 로 구분한다.

```
outline_template:
{outline_template}
```

2



사용자가 피드백을 추가로 제공할 수 있도록 논리적인 흐름과 주요 목차 아이디어를 제안하라.

```
"""
)

시스템 프롬프트와 모델 연결
content_strategist_chain = content_strategist_system_prompt | llm | StrOutputParser()

user_request = state.get("user_request", "") ③
messages = state["messages"] # 상태에서 메시지 가져오기
outline = get_outline(current_path) # 저장된 목차 가져오기
템플릿 이용하기
with open(f"{current_path}/templates/outline_template.md", "r", encoding='utf-8')
as f:
 outline_template = f.read()

입력값 정의
inputs = {
 "user_request": user_request,
 "task": task, ③
 "messages": messages,
 "outline": outline,
 "references": state.get("references", {"queries": [], "docs": []}),
 "outline_template": outline_template ④
}

목차 작성
gathered = ''
for chunk in content_strategist_chain.stream(inputs):
 gathered += chunk
 print(chunk, end='')

print()

save_outline(current_path, gathered) # 목차 저장

if '-----: DONE :-----' in gathered:
 review = gathered.split('-----: DONE :-----')[1]
else:
 review = gathered[-200:]
```

```

메시지 추가
content_strategist_message = f"[Content Strategist] 목차 작성 완료: outline
작성 완료\n {review}"
print(content_strategist_message)
messages.append(AIMessage(content_strategist_message))

task_history = state.get("task_history", []) — ① 로 이동
최근 task 작업 완료(done) 처리하기
if task_history[-1].agent != "content_strategist":
raise ValueError(f"Content Strategist가 아닌 agent가 목차 작성을 시도하고 있습니다.
\n {task_history[-1]}")

task_history[-1].done = True
task_history[-1].done_at = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

new_task = Task(
 agent="communicator",
 description="AI 팀의 진행 상황을 사용자에게 보고하고, 사용자의 의견을 파악하기 위해 대화를 나눈다."
)

task_history.append(new_task)
print(new_task)

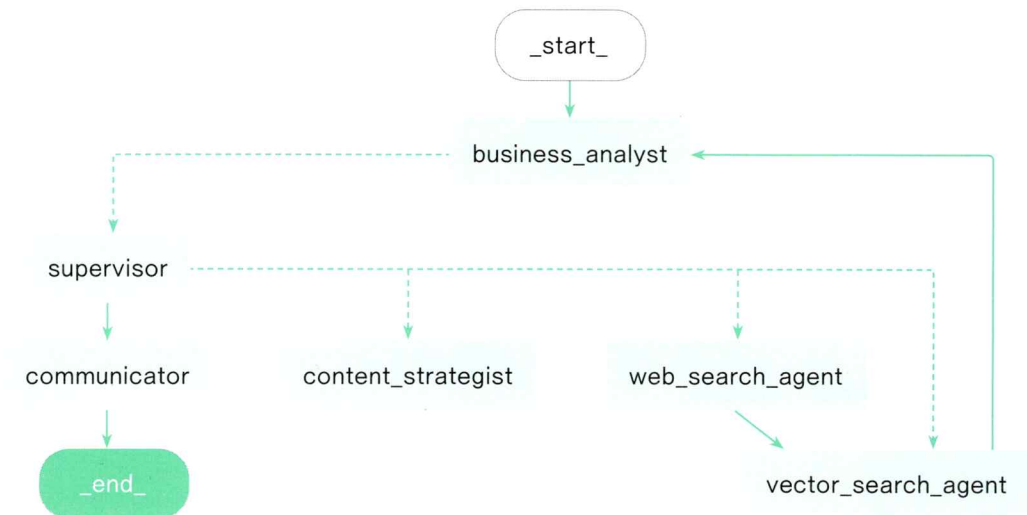
return {
 "messages": messages,
 "task_history": task_history
}
(... 생략 ...)
```

- ① content\_strategist 함수 아래쪽에 있던 작업 이력을 가져오는 코드를 위쪽으로 옮깁니다. if task\_history[-1].agent를 task = task\_history[-1]로 수정하고 if task.agent로 변경합니다. 이렇게 수정하면 이 노드가 실행되자마자 마지막 작업이 content\_strategist인지 확인하여 의도하지 않은 노드가 실행된 경우를 확인할 수 있습니다. 그리고 현재 해야 할 작업에 대한 설명이 포함된 task 객체를 활용하도록 content\_strategist\_system\_prompt를 수정할 예정입니다.
- ② 기존의 시스템 프롬프트에 요구 사항을 더 구체적으로 추가합니다. 앞서 작성한 템플릿을 이용하고, 출력 방식도 자세하게 설명합니다.
- ③ 사용자 요구 사항인 user\_request를 state에서 가져옵니다. user\_request는 목차를 생성하는 content\_strategist가 현재 사용자의 요구 사항을 염두에 두고 작업하도록 유도하는 장치로 business\_analyst가 생성합니다.
- ④ 프롬프트에 앞서 만든 마크다운 형식의 템플릿인 outline\_template.md 파일을 활용합니다. 목차를 작성할 때 지켜야 할 사항과 템플릿을 마크다운 문서로 적어 놓았습니다. 이 템플릿 파일을 읽어서 프롬프트에 포함 시킵니다.

- 5 목차를 작성하는 템플릿 파일에는 목차를 작성하고 그 후기를 적으라는 내용이 포함되어 있습니다. 이 값을 가져와서 작업 후기를 `messages`에 추가합니다. 이렇게 하면 `content_strategist`가 작업한 후기를 대화 기록(`messages`)에 추가해서 다른 에이전트들이 지금 무슨 일을 어떻게 진행하는지를 파악할 수 있습니다.

## 스스로 판단하고 작업하는 멀티에이전트

지금까지는 각 에이전트가 작업을 마칠 때마다 커뮤니케이터 에이전트인 `communicator`와 연결되어 사용자에게 진행 상황을 보고하고 다음 작업을 물어보았습니다. 이제 사용자에게 다음 작업을 물어보기 전에 비즈니스 분석가 에이전트 `business_analyst`가 현재 상태를 파악하고, 확인이 필요한 경우에만 사용자에게 문의하도록 시스템을 수정하겠습니다. 이제 멀티에이전트 구조가 다음 그림과 같이 변경됩니다.



현재 상황을 파악하고 작업을 판단하는 멀티에이전트의 그래프 구조

## Do it! 실습 스스로 판단하고 작업하는 멀티에이전트 시스템 만들기

■ 결과 파일: sec02/book\_writer.py, models.py

1. `business_analyst`가 현재 상황을 파악해 이후 작업을 판단하도록 엣지의 연결 구조를 수정합니다. `content_strategist`와 `vector_search_agent`의 작업이 종료되면 결과가 다시 `business_analyst`로 돌아옵니다. 연결 구조를 이렇게 구성하면 각 AI 에이전트가 작업을 마친 후에 `business_analyst`가 전체 흐름을 파악하고 필요한 작업을 계속 이어 나갈 수 있습니다.

목차 작성 혹은 벡터 검색 후 `business_analyst` 노드로 연결하기

■ book\_writer.py

```
엣지
graph_builder.add_edge(START, "business_analyst")
graph_builder.add_edge("business_analyst", "supervisor")
graph_builder.add_conditional_edges(
 "supervisor",
 supervisor_router,
 {
 "content_strategist": "content_strategist",
 "communicator": "communicator",
 "vector_search_agent": "vector_search_agent",
 "web_search_agent": "web_search_agent"
 }
)
graph_builder.add_edge("content_strategist", "business_analyst")
graph_builder.add_edge("web_search_agent", "vector_search_agent")
graph_builder.add_edge("vector_search_agent", "business_analyst")
graph_builder.add_edge("communicator", END)

graph = graph_builder.compile()
```

2. 이전에는 목차를 작성하는 `content_strategist` 뒤에 `communicator`가 연결되었지만 이제는 작업을 이어가기 위해 `business_analyst`가 연결됩니다. 따라서 `content_strategist`에서 새로운 작업을 만드는 코드를 삭제합니다.

`content_strategist`에서 task 생성 없애기

■ book\_writer.py

```
(... 생략 ...)
def content_strategist(state: State):
 (... 생략 ...)
```

```

다음 작업이 communicator로 사용자와 대화하는 것이므로 새 작업 추가
new_task = Task(
agent="communicator",
done=False,
description="AI 팀의 진행 상황을 사용자에게 보고하고, 사용자의 의견을 파악하기 위해 대
화를 나눈다.",
done_at=""
)
task_history.append(new_task)
print(new_task)

현재 state를 업데이트
return {
 "messages": messages,
 "task_history": task_history
}

```

3. 지금까지는 supervisor가 벡터 검색을 먼저 수행한 뒤 웹 검색으로 검색 결과를 보강하도록 관리하는 개념이 없었습니다. supervisor가 벡터 검색을 먼저 지시하고 그 결과가 부족할 경우에만 웹 검색을 진행하게 하는 내용을 프롬프트에 추가합니다.

벡터 검색 결과가 만족스럽지 않을 때 웹 검색을 하도록 수정하기

book\_writer.py

```

(... 생략 ...)
def supervisor(state: State):
 print("\n\n===== SUPERVISOR =====")

 # 시스템 프롬프트 정의
 supervisor_system_prompt = PromptTemplate.from_template(
 """
 너는 AI 팀의 supervisor로서 AI 팀의 작업을 관리하고 지도한다.
 사용자가 원하는 책을 써야 한다는 최종 목표를 염두에 두고,
 사용자의 요구를 달성하기 위해 현재 해야 할 일이 무엇인지 결정한다.

 supervisor가 활용할 수 있는 agent는 다음과 같다.
 - content_strategist: 사용자의 요구 사항이 명확해졌을 때 사용한다. AI 팀의 콘텐츠 전략
을 결정하고, 전체 책의 목차(outline)를 작성한다.
 - communicator: AI 팀에서 해야 할 일을 스스로 판단할 수 없을 때 사용한다. 사용자에게 진
행 상황을 사용자에게 보고하고, 다음 지시를 물어본다.
 - web_search_agent: vector_search_agent를 시도하고, 검색 결과(references)에 필요한
정보가 부족한 경우 사용한다. 웹 검색을 통해 해당 정보를 벡터 DB에 보강한다.
 - vector_search_agent: 목차 작성에 필요한 자료를 확보하기 위해 벡터 DB 검색을 한다.
 """
)

```



아래 내용을 고려하여, 현재 해야 할 일이 무엇인지, 사용할 수 있는 agent가 무엇인지 단답으로 말하라.

```

previous_outline: {outline}

messages:
{messages}
"""

)

(... 생략 ...)
```

4. 멀티에이전트 시스템은 Task에 영향을 받으므로 models.py 파일에 있는 Task의 Field 설명을 변경 사항에 맞게 수정합니다.

#### Task의 agent 설명 수정하기

models.py

(... 생략 ...)

```
class Task(BaseModel):
 agent: Literal[
 "content_strategist",
 "communicator",
 "web_search_agent",
 "vector_search_agent",
] = Field(
 ...,
 description="""
 작업을 수행하는 agent의 종류.
 - content_strategist: 콘텐츠 전략을 수립하는 작업을 수행한다. 사용자의 요구 사항이 명확
 해졌을 때 사용한다. AI 팀의 콘텐츠 전략을 결정하고, 전체 책의 목차(outline)를 작성한다.
 - communicator: AI 팀에서 해야 할 일을 스스로 판단할 수 없을 때 사용한다. 사용자에게 진
 행 상황을 보고하고, 다음 지시를 물어본다.
 - web_search_agent: 웹 검색으로 목차(outline) 작성에 필요한 정보를 확보한다.
 - vector_search_agent: 벡터 DB 검색으로 목차(outline) 작성에 필요한 정보를 확보한다.
 """
)

(... 생략 ...)
```

이 코드를 실행해 보면 사용자의 요구 사항에 맞춰 멀티에이전트가 자동으로 목차 생성을 진행합니다. 다음은 'HYBE와 JYP의 CEO, 경영 전략, 기업 문화에 대해 비교하는 내용으로 목차를 써줘.'라고 요청했을 때 멀티에이전트가 독립적으로 작업을 진행해 실제 자료를 활용하여 생성한 결과물입니다.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <h3>HYBE와 JYP: 두 거인의 비교와 탐구</h3> <p>보고서 부제목: K-POP의 선두주자, 그들의 혁신과 전략을 파헤치다</p> <h4>Chapter 1: 기업 개요 및 역사</h4> <ul style="list-style-type: none"> <li>Chapter 목적: HYBE와 JYP의 기본적인 역사를 소개하고 두 기업의 성장 과정을 이해합니다.</li> </ul> <h5>Section 1.1: HYBE의 역사와 성장</h5> <ul style="list-style-type: none"> <li>Section 목적: HYBE의 설립 배경과 주요 성장 과정을 파악합니다.</li> <li>Section 내용: <ul style="list-style-type: none"> <li>HYBE 설립 배경</li> <li>주요 아티스트와 히트작</li> <li>성장의 주요 전환점</li> </ul> </li> <li>참고문헌: <ol style="list-style-type: none"> <li>HYBE 공식 웹사이트: HYBE의 회사 연혁 및 주요 아티스트 정보</li> <li>관련 뉴스 기사: HYBE의 주요 성장 이정표</li> </ol> </li> </ul> <h5>Section 1.2: JYP의 역사와 성장</h5> <ul style="list-style-type: none"> <li>Section 목적: JYP의 설립 배경과 주요 성장 과정을 파악합니다.</li> <li>Section 내용: <ul style="list-style-type: none"> <li>JYP 설립 배경</li> <li>주요 아티스트와 히트작</li> <li>성장의 주요 전환점</li> </ul> </li> <li>참고문헌: <ol style="list-style-type: none"> <li>JYP 공식 웹사이트: JYP의 회사 연혁 및 주요 아티스트 정보</li> <li>관련 뉴스 기사: JYP의 주요 성장 이정표</li> </ol> </li> </ul> <p>---CHAPTER DIVIDER---</p> <h4>Chapter 2: CEO와 리더십 스타일</h4> <ul style="list-style-type: none"> <li>Chapter 목적: HYBE와 JYP의 CEO의 리더십 스타일을 비교하여 경영 철학의 차이를 분석합니다.</li> </ul> <h5>Section 2.1: 방시혁과 이재상의 리더십</h5> <ul style="list-style-type: none"> <li>Section 목적: HYBE의 창립자 방시혁 및 현재 CEO 이재상의 경영 철학과 리더십 스타일을 분석합니다.</li> <li>Section 내용: <ul style="list-style-type: none"> <li>방시혁의 경영 철학</li> <li>이재상의 리더십 스타일 및 영향력</li> <li>주요 경영 전략 사례</li> </ul> </li> <li>참고문헌: <ol style="list-style-type: none"> <li>방시혁 인터뷰 기사: 방시혁의 경영 철학에 대한 인터뷰 내용</li> <li>HYBE 리더십 관련 문서: 이재상의 리더십 스타일 분석</li> </ol> </li> </ul> <h5>Section 2.2: 박진영의 리더십</h5> <ul style="list-style-type: none"> <li>Section 목적: JYP의 창립자 박진영의 경영 철학과 리더십 스타일을 분석합니다.</li> <li>Section 내용: <ul style="list-style-type: none"> <li>박진영의 경영 철학</li> <li>리더십 스타일 및 영향력</li> <li>주요 경영 전략 사례</li> </ul> </li> <li>참고문헌: <ol style="list-style-type: none"> <li>박진영 인터뷰 기사: 박진영의 경영 철학에 대한 인터뷰 내용</li> <li>JYP 리더십 관련 문서: 박진영의 리더십 스타일 분석</li> </ol> </li> </ul> <p>---CHAPTER DIVIDER---</p> <h4>Chapter 3: 경영 전략 비교</h4> <ul style="list-style-type: none"> <li>Chapter 목적: 두 기업의 경영 전략을 비교하고 분석하여 차이점과 공통점을 도출합니다.</li> </ul> <h5>Section 3.1: HYBE의 경영 전략</h5> <ul style="list-style-type: none"> <li>Section 목적: HYBE의 독특한 경영 전략을 분석합니다.</li> <li>Section 내용: <ul style="list-style-type: none"> <li>글로벌 전략 및 확장</li> <li>디지털 플랫폼 활용</li> <li>브랜드 가치 강화 전략</li> </ul> </li> <li>참고문헌: <ol style="list-style-type: none"> <li>HYBE 경영 전략 문서: HYBE의 글로벌 전략 분석</li> </ol> </li> </ul> <h5>Section 3.2: JYP의 경영 전략</h5> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

  |

현재 코드는 AI 에이전트들이 목차를 끊임없이 개선하려고 할 수도 있습니다. 슈퍼바이저 에이전트가 커뮤니케이터 에이전트에게 일을 주지 않으면 작업이 계속되기 때문입니다. 이렇게 되면 결과물의 품질을 개선하지 못하는데도 오픈AI의 API 토큰을 불필요하게 소모합니다. 이런 경우 다음과 같은 오류 메시지와 함께 자동으로 중단되기도 하지만 **Ctrl** + **C**를 눌러 중간에 작업을 중단시킬 수도 있습니다.

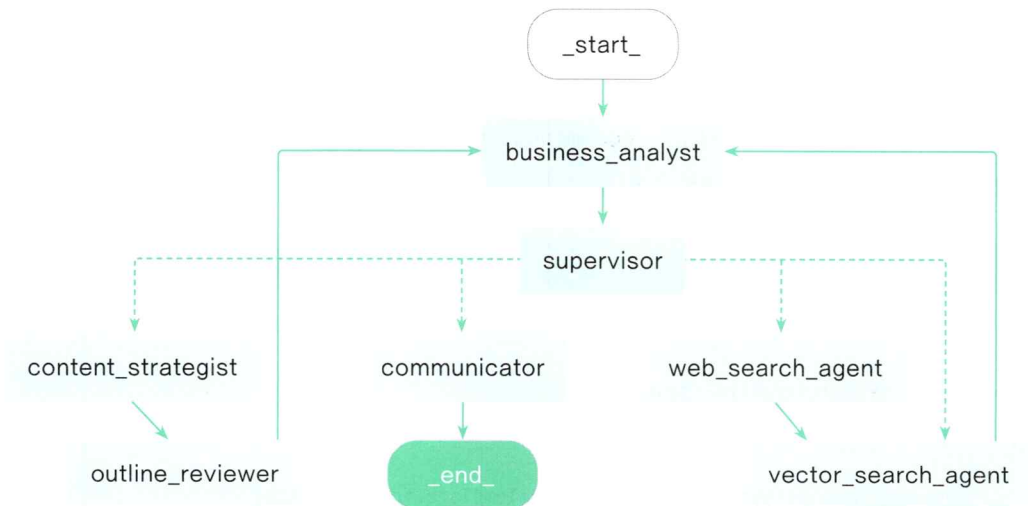
```
langgraph.errors.GraphRecursionError: Recursion limit of 25 reached without hitting a stop condition. You can increase the limit by setting the `recursion_limit` config key. For troubleshooting, visit: https://python.langchain.com/docs/troubleshooting/errors/GRAPH_RECURSION_LIMIT
```

## 15-3 스스로 리뷰하고 수정하는 에이전트로 발전시키기

이제 AI 에이전트들이 회의하여 작성한 목차를 보면서 사람이 검토하고 수정 방향을 알려 줄 수 있습니다. 그런데 인공지능이 스스로 내용을 검토하고 수정할 수 있다면 더욱 편리하겠죠? 작업한 결과를 확인하는 목차 리뷰 에이전트를 만들어 사람이 확인하지 않아도 AI 에이전트가 파악할 수 있는 문제점은 스스로 해결하고 더 발전된 결과를 제공할 수 있도록 만들어 보겠습니다.

### 목차 리뷰 에이전트

콘텐츠 전략가 에이전트 `content_strategist`가 목차를 잘 작성했는지 판단하는 역할을 하는 목차 검토 에이전트 `outline_reviewer`를 만들겠습니다. `outline_reviewer`가 목차를 보고 ‘이런 부분은 이렇게 수정하면 좋겠다’고 조언하면 `business_analyst`가 어떤 작업을 할지 판단해서 다시 작업을 진행하는 방식입니다.



목차를 분석하는 `outline_reviewer` 에이전트

## Do it! 실습 목차 조언 항목 추가하고 business\_analyst에 반영하기

■ 결과 파일: sec03/book\_writer\_0.py

목차 검토 에이전트 outline\_reviewer가 목차를 분석하고 조언하는 내용을 상태에 담아 비즈니스 분석가 에이전트 business\_analyst가 활용할 수 있도록 만들어 보겠습니다. 지금까지 계속 작업했던 book\_writer.py 파일을 수정하면 됩니다.

1. 목차를 리뷰하는 에이전트를 만들기 전에, State에 이 에이전트가 목차를 분석해 조언한 내용을 담아 둘 ai\_recommendation 변수를 새로 만듭니다.

State에 AI의 조언을 담아 둘 변수 추가하기

■ book\_writer.py

```
(... 생략 ...)
```

```
상태 정의
class State(TypedDict):
 messages: List[AnyMessage | str]
 task_history: List[Task]
 references: dict # RAG 에이전트에서 검색한 정보를 저장하는 변수
 user_request: str # 사용자의 요구 사항을 저장하는 변수
 ai_recommendation: str # AI의 추천을 저장하는 변수

(... 생략 ...)
```

2. 이제 outline\_reviewer가 ai\_recommendation을 채워 놓았다고 가정하고 그 값을 이용해 business\_analyst가 판단하도록 수정합니다.

outline\_reviewer의 조언을 받아들이 수 있도록 business\_analyst 수정하기

■ book\_writer.py

```
(... 생략 ...)
```

```
def business_analyst(state: State):
 print("\n\n===== BUSINESS ANALYST =====")

 business_analyst_system_prompt = PromptTemplate.from_template(
 """
 너는 책을 쓰는 AI 팀의 비즈니스 애널리스트로서,
 AI 팀의 진행 상황과 "사용자 요구 사항"을 토대로,
 현 시점에서 'ai_recommendation'과 최근 사용자의 발언을 바탕으로 요구 사항이 무엇인지 판단
 한다. —①
 지난 요구 사항이 달성되었는지 판단하고, 현 시점에서 어떤 작업을 해야 하는지 결정한다.
 """
)
```

다음과 같은 템플릿 형태로 반환한다.

```

- 목표: 0000 \n 방법: 0000

```

-----  
\*AI 추천(ai\_recommendation)\* : {ai\_recommendation}—①

-----  
사용자 최근 발언: {user\_last\_comment}

-----  
참고 자료: {references}

-----  
목차 (outline): {outline}

-----  
"messages": {messages}

""

)

(... 생략 ...)

inputs = {

  "ai\_recommendation": state.get("ai\_recommendation", None),—②

  "references": state.get("references", {"queries": [], "docs": []}),

  "outline": get\_outline(current\_path),

  "messages": messages,

  "user\_last\_comment": user\_last\_comment

}

(... 생략 ...)

return {

  "messages": messages,

  "user\_request": user\_request,

  "ai\_recommendation": ""—③

}

(... 생략 ...)

- ① '사용자의 이전 요구 사항'을 이용해서 판단하라고 되어 있던 시스템 프롬프트를 ai\_recommendation을 바탕으로 판단하도록 수정합니다.
- ② 프롬프트가 수정되었으므로 이에 맞춰 inputs도 수정합니다. State에 ai\_recommendation을 추가했으므로 이 값을 이용합니다.



- 3 ai\_recommendation은 business\_analyst가 사용자의 의도와 현재 상황을 바탕으로 무엇을 해야 할지 판단하는 데만 필요하므로 다른 부분에 실수로 영향을 끼치지 않도록 처리합니다. state에서 ai\_recommendation을 빈 값으로 만들기 위해 ""로 업데이트합니다.

### Do it! 실습 목차를 검토하는 outline\_reviewer 만들기

■ 결과 파일: sec03/book\_writer\_1.py

이제 목차를 리뷰하는 에이전트 outline\_reviewer를 만들겠습니다. outline\_reviewer는 content\_strategist가 목차를 생성한 뒤 자동으로 실행될 예정입니다.

#### outline\_reviewer 만들기

■ book\_writer.py

```
목차를 작성하는 노드(agent)
def content_strategist(state: State):
 (... 생략 ...)
```

```
def outline_reviewer(state: State):
 print("\n\n===== OUTLINE REVIEWER =====")

 outline_reviewer_system_prompt = PromptTemplate.from_template(
 """
 너는 AI팀의 목차 리뷰어로서, AI팀이 작성한 목차(outline)를 검토하고 문제점을 지적한다.

 - outline이 사용자의 요구사항을 충족시키는지 여부
 - outline의 논리적인 흐름이 적절한지 여부
 - 근거에 기반하지 않은 내용이 있는지 여부
 - 주어진 참고자료(references)를 충분히 활용했는지 여부
 - 참고자료가 충분한지, 혹은 잘못된 참고자료가 있는지 여부
 - example.com 같은 더미 URL이 있는지 여부:
 - 실제 페이지 URL이 아닌 대표 URL로 되어 있는 경우 삭제 해야 함: 어떤 URL이
 삭제되어야 하는지 명시하라.
 - 기타 리뷰 사항

 그 분석 결과를 설명하고, 다음에 어떤 작업을 하면 좋을지 제안하라.

 - 분석 결과: outline이 사용자의 요구사항을 충족시키는지 여부
 - 제안 사항: (vector_search_agent, communicator 중 어떤 agent를 호출할지)

 user_request: {user_request}

 references: {references}
```

```

outline: {outline}

messages: {messages}
"""
)

```

1

```

user_request = state.get("user_request", None)
outline = get_outline(current_path)
references = state.get("references", {"queries": [], "docs": []})
messages = state.get("messages", [])

inputs = {
 "user_request": user_request,
 "outline": outline,
 "references": references,
 "messages": messages
}

```

2

```

시스템 프롬프트와 모델을 연결
outline_reviewer_chain = outline_reviewer_system_prompt | llm

review = outline_reviewer_chain.stream(inputs)

gathered = None

for chunk in review:
 print(chunk.content, end='')

 if gathered is None:
 gathered = chunk
 else:
 gathered += chunk

```

3

```

if '[OUTLINE REVIEW AGENT]' not in gathered.content:
 gathered.content = f"[OUTLINE REVIEW AGENT] {gathered.content}"

```

4

```

print(gathered.content)
messages.append(gathered)

ai_recommendation = gathered.content

```

5

```

return {"messages": messages, "ai_recommendation": ai_recommendation}

```

6

- ① `outline_reviewer`의 시스템 프롬프트는 지금까지 `content_strategist`가 만든 목차의 문제점이 무엇인지 파악하는데 초점을 맞춰야 합니다. 목차 구성이 사용자의 요구 사항에 맞는지, 근거 없이 작성하지 않았는지, 참고 자료를 충분히 활용했는지 등을 체크리스트로 확인하도록 합니다. 예를 들어 목차를 만들 때 참고하는 웹 페이지 중 'example.com' 같은 더미 URL이나 가짜 URL이 남아 있는지 확인합니다.
- ② `outline_reviewer`의 프롬프트에 맞게 `inputs`를 설정합니다. `content_strategist`에서 생성된 목차와 관련 정보를 전달해야 합니다.
- ③ 목차 리뷰가 길어질 수 있으므로 출력 방식을 `.invoke`가 아닌 `.stream` 방식으로 설정하여 리뷰 결과를 터미널 창에서 스트림 방식으로 확인할 수 있게 합니다.
- ④ 목차 리뷰가 완료되면 이 내용을 기존 대화 목록인 `messages`에 `outline_reviewer`의 작업 후기로 추가합니다.
- ⑤ 리뷰 결과는 `ai_recommendation`의 값으로도 활용합니다.
- ⑥ 수정한 `state`를 업데이트하기 위해 이 값을 반환합니다.

### Do it! 실습 벡터 검색 에이전트도 비즈니스 분석가 에이전트에게 조언하도록 구성하기

■ 결과 파일: `sec03/book_writer_2.py`

벡터 검색 에이전트 `vector_search_agent`도 비즈니스 분석가 에이전트 `business_analyzer`에게 조언할 수 있게 수정해 보겠습니다. 앞에서 설계한 그래프를 떠올려 보면 벡터 DB에서 관련 문서를 검색하는 `vector_search_agent`도 작업이 종료되면 `business_analyzer`로 연결됩니다. 따라서 기존 코드에서 작업을 종료한 후 커뮤니케이터 에이전트 `communicator`로 연결되는 작업을 생성하는 대신 정보를 `business_analyzer`로 전달할 수 있도록 수정해 보겠습니다.

1. `book_writer.py`에서 목차 작성이 종료된 후 `communicator`로 연결되는 작업을 생성하는 코드를 삭제합니다. 그 대신 `ai_recommendation`을 작성할 수 있도록 수정합니다. 벡터 검색 결과로 자료가 충분히 모였다면 목차를 수정하거나 개선할 수 있도록 `content_strategist`를 추천하는 메시지를 작성합니다. 만약 벡터 검색으로 자료를 충분히 찾지 못했다면 AI 에이전트들이 알아서 다음 업무를 선택할 것입니다.

(... 생략 ...)

```
def vector_search_agent(state: State):
```

```
 print("\n\n===== VECTOR SEARCH AGENT =====")
```

(... 생략 ...)

```
 # task 완료
```

```
 tasks[-1].done = True
```

```
 tasks[-1].done_at = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
```

삭제

```
 # communicator로 보내는 Task 생성 코드 삭제
 # new_task = Task(
 # agent="communicator",
 # done=False,
 # description="AI팀의 진행상황을 사용자에게 보고하고, 사용자의 의견을 파악하기 위한 대화
 # 를 나눈다"
 # done_at="",
 #)

 # tasks.append(new_task)
```

```
msg_str = f"[VECTOR SEARCH AGENT] 다음 질문에 대한 검색 완료: {queries}"
```

```
message = AIMessage(msg_str)
```

```
print(msg_str)
```

```
messages.append(message)
```

```
communicator
```

ai\_recommendation = "현재 참고자료(references)가 목차(outline)를 개선하는 데 충분한지 확인하라. 충분하다면 content\_strategist로 목차 작성을 하라."

```
return {
```

```
 "messages": messages,
```

```
 "task_history": tasks,
```

```
 "references": references,
```

```
 "ai_recommendation": ai_recommendation
```

```
}
```

2. 새로 만든 목차 리뷰 에이전트 `outline_reviewer`를 노드로 추가하고 `edge`를 시스템의 흐름에 맞게 설정합니다. `content_strategist`의 작업이 종료되면 곧바로 `outline_reviewer`로 작업이 넘어갑니다. 그리고 `outline_reviewer`의 작업이 종료되면 `business_analyst`로 전달되어 후속 작업이 진행되도록 처리합니다.

#### 그래프 수정하기

book\_writer.py

```
(... 생략 ...)
상태 그래프 정의
graph_builder = StateGraph(State)

노드
graph_builder.add_node("business_analyst", business_analyst)
graph_builder.add_node("supervisor", supervisor)
graph_builder.add_node("communicator", communicator)
graph_builder.add_node("content_strategist", content_strategist)
graph_builder.add_node("outline_reviewer", outline_reviewer)
graph_builder.add_node("vector_search_agent", vector_search_agent)
graph_builder.add_node("web_search_agent", web_search_agent)

엣지
graph_builder.add_edge(START, "business_analyst")
graph_builder.add_edge("business_analyst", "supervisor")
graph_builder.add_conditional_edges(
 "supervisor",
 supervisor_router,
 {
 "content_strategist": "content_strategist",
 "communicator": "communicator",
 "vector_search_agent": "vector_search_agent",
 "web_search_agent": "web_search_agent"
 }
)
graph_builder.add_edge("content_strategist", "outline_reviewer")
graph_builder.add_edge("outline_reviewer", "business_analyst")
graph_builder.add_edge("web_search_agent", "vector_search_agent")
graph_builder.add_edge("vector_search_agent", "business_analyst")
graph_builder.add_edge("communicator", END)

graph = graph_builder.compile()
(... 생략 ...)
```



현재 상태에서 목차가 더 잘 생성되는지 확인하기 위해 기존의 outline.md 파일을 삭제하고 다시 실행합니다. 터미널 창에 출력되는 로그가 많아서 다 발췌할 수는 없지만 다음과 같이 적절하게 분석하면서 다음 작업을 잘 선택함을 확인할 수 있습니다.

===== OUTLINE REVIEWER =====

### 분석 결과:

1. \*\*사용자 요구사항 충족 여부:\*\*

- 사용자의 요구사항은 HYBE와 JYP를 비교하는 책을 작성하는 것으로, CEO, 경영 전략, 기업 문화, 실적에 대한 심도 있는 분석을 포함해야 합니다. 현재 목차는 이러한 요구사항을 충실히 반영하고 있습니다. 각 챕터는 사용자가 원하는 주요 분석 영역을 잘 다루고 있습니다.

2. \*\*논리적 흐름의 적절성:\*\*

- 목차는 CEO와 리더십에서 시작하여 경영 전략, 기업 문화, 실적 분석으로 이어지는 구조로, 논리적인 흐름이 적절합니다. 각 섹션이 명확한 분석 목표를 가지고 있어 독자가 두 기업을 심도 있게 비교할 수 있도록 구성되어 있습니다.

3. \*\*근거에 기반하지 않은 내용 여부:\*\*

- 현재 목차에서는 구체적인 근거가 제시되어 있지 않으며, 각 섹션에 대한 세부적인 연구와 데이터를 활용해야 할 필요가 있습니다.

4. \*\*참고자료 활용 여부 및 충분성:\*\*

- 제공된 목차에서는 'example.com'과 같은 더미 URL이 사용되고 있어 참고자료가 실제로 활용되지 않았습니다. 이는 삭제되어야 합니다. 현재 문서에서는 참고자료가 전혀 없으므로 추가적인 자료 수집이 반드시 필요합니다.

5. \*\*추가 개선점:\*\*

- 목차 작성 후기에 언급된 바와 같이, 각 챕터의 세부 참고자료를 보다 풍부하게 준비할 필요가 있습니다. 이를 위해 추가적인 자료 수집이 필요하며, 이는 책의 신뢰성을 높이는 데 필수적입니다.

### 제안 사항:

- \*\*Agent 호출 제안:\*\*

- `vector\_search\_agent`: 이 에이전트를 호출하여 HYBE와 JYP에 대한 더 많은 참고 자료를 수집하고, 각 섹션의 분석에 필요한 구체적인 데이터와 연구를 확보합니다. 이를 통해 목차의 각 챕터가 보다 구체적이고 근거 기반의 내용을 갖출 수 있도록 지원합니다.

(... 생략 ...)

현재 코드는 멀티에이전트가 더 이상 어떻게 해야 할지 모르는 상황에서 사용자에게 질문을 하도록 되어 있지만, 스스로 해결을 해보려고 사용자에게 물어보지 않는 경우가 자주 있습니다. 이런 상황에서는 결과물의 품질은 높아지지 않으므로 중간에 **[Ctrl] + [C]**를 눌러 종료하기 바랍니다. 이 문제의 해결 방법은 다음 실습 예제에서 다룹니다.

## Do it! 실습 무한 루프 방지하기

결과 파일: sec03/book\_writer.py

현재 구조에서는 AI 종종 에이전트끼리 계속 작업을 주고받으면서 무한히 개선하려 하는 경우가 있습니다. 이런 경우 크게 개선되는 요소는 없이 GPT 토큰과 타빌리 검색 쿼리만 소모하게 됩니다. 시간도 오래 걸리고 사용자의 의견을 추가로 반영할 시간도 주지 않는 문제가 있습니다.

결국에는 재귀 한계<sup>recursion limit</sup>에 걸려 다음과 같이 오류를 뱉습니다. 이 오류는 토큰을 무한히 소모하는 사람들을 위해 랭그래프에서 만들어 둔 안전 장치입니다. 최대 25바퀴 이상 돌지 못하게 기본으로 설정되어 있고, 필요하다면 시도 횟수를 더 늘릴 수 있는 옵션도 제공합니다.

```
File "C:\github\gpt_agent_2024_book\venv\Lib\site-packages\langgraph\pregel_init_.py", line 1632, in stream
 raise GraphRecursionError(msg)
langgraph.errors.GraphRecursionError: Recursion limit of 25 reached without hitting a stop condition. You can increase the limit by setting the `recursion_limit` config key. For troubleshooting, visit: https://python.langchain.com/docs/troubleshooting/errors/GRAPH_RECURSION_LIMIT
```

1. 무한 루프에 빠지지 않도록 supervisor를 몇 번 호출했는지 기록하는 변수를 정수 타입으로 추가합니다.

무한 루프에 빠지지 않도록 supervisor를 호출한 횟수 저장하기

book\_writer.py

```
상태 정의
class State(TypedDict):
 messages: List[AnyMessage | str]
 task_history: List[Task]
 references: dict # RAG 에이전트에서 검색한 정보를 저장하는 변수
 user_request: str # 사용자의 요구사항을 저장하는 변수
 ai_recommendation: str # AI의 추천을 저장하는 변수
 supervisor_call_count: int # supervisor 호출 횟수를 저장하는 변수
```

2. supervisor가 2회 이하로 호출되었다면 원래대로 다음 작업을 적절하게 선택하도록 하고 2회를 초과할 경우 사용자와 대화하는 communicator가 다음 작업이 되도록 지정합니다. 그리고 state 값을 반환할 때는 supervisor\_call\_count에 1을 더한 상태로 업데이트하여 반복 호출을 추적할 수 있도록 합니다.

```

(... 생략 ...)
def supervisor(state: State):
 print("\n\n===== SUPERVISOR =====")

 (... 생략 ...)

 inputs = {
 "messages": messages,
 "outline": get_outline(current_path)
 }

 supervisor_call_count = state.get("supervisor_call_count", 0)

 if supervisor_call_count > 2:
 print("Supervisor 호출 횟수 초과: Communicator 호출")
 task = Task(
 agent="communicator",
 done=False,
 description="supervisor 호출 횟수를 초과했으므로, 현재까지의 진행 상황을 사용자에게
 보고한다. ",
 done_at="",
)
 else:
 task = supervisor_chain.invoke(inputs)

 task_history = state.get("task_history", []) # 작업 이력 가져오기
 task_history.append(task) # 작업 이력에 추가

 supervisor_message = AIMessage(f"[Supervisor] {task}")
 messages.append(supervisor_message)
 print(supervisor_message.content)

 return {
 "messages": messages,
 "task_history": task_history,
 "supervisor_call_count": supervisor_call_count + 1
 }
(... 생략 ...)

```



3. 사용자에게 진행 상황을 보고하는 커뮤니케이터 에이전트 `communicator`가 작동되었을 때는 `supervisor_call_count`를 0으로 리셋해야 합니다. 그래야 또다시 `supervisor`의 호출이 2회를 초과할 때까지는 AI 에이전트들끼리 알아서 작업할 수 있으니까요.

```

커뮤니케이터가 사용자에게 보고하면 supervisor_call_count를 0으로 리셋하기
book_writer.py

(... 생략 ...)
사용자와 대화할 노드(agent): communicator
def communicator(state: State):
 print("\n\n===== COMMUNICATOR =====")

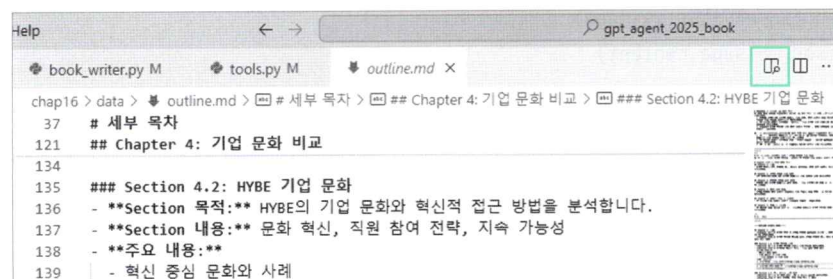
 (... 생략 ...)

 return {
 "messages": messages,
 "task_history": task_history,
 "supervisor_call_count": 0
 }
(... 생략 ...)

```

이 코드를 실행을 해보니 작업을 열심히 진행하다가 가끔씩 사용자에게 돌아와 진행 상황을 묻는 형태로 개선되었습니다. 다음은 ‘HYBE와 JYP를 비교하는 목차를 써줘. CEO, 경영 전략, 기업 문화, 특성에 대해 설명해 줘.’라고만 입력한 결과입니다.

출력된 마크다운 파일에서 오른쪽 상단의 마크다운 렌더링 버튼을 클릭하면 마크다운 문서가 보기 좋게 렌더링되어서 나옵니다.



## Chapter 1: 서론

- **Chapter 목적:** K-POP 산업 내 JYP와 HYBE의 중요성을 소개하고, 책의 분석 목적을 정의합니다.
- **Chapter 내용:** K-POP 산업의 역사와 성장, JYP와 HYBE의 현재 위치 설명, 분석 목표 설정

### Section 1.1: K-POP 산업의 개요

- **Section 목적:** K-POP 산업의 발전과 주요 특징을 설명합니다.
- **Section 내용:** K-POP의 역사적 배경, 글로벌 성장 동향 분석
- **주요 내용:**
  - 역사적 맥락 설명
  - 주요 트렌드 및 시장 규모
  - 글로벌 영향력 및 팬덤 문화
- **참고문헌:**
  1. "K-POP의 글로벌 확산"
  2. "K-POP 산업 동향 보고서"

결과를 살펴보면 목차가 그럴싸하게 만들어졌습니다. 이 시스템에 벡터 검색한 결과를 리뷰하는 AI 에이전트를 추가하거나 작성된 목차를 바탕으로 실제 책 내용을 작성하는 AI 에이전트를 추가할 수도 있습니다. 어떤 방식으로 멀티에이전트 시스템을 개선할지 상상하고 직접 구현해 보세요.